

# Support Vector Machines

Sandro Cumani

sandro.cumani@polito.it

Politecnico di Torino

# Support Vector Machines

Let's consider again a binary classification problem

We have shown that logistic regression provides a linear classification rule that maximizes the log-probability of class assignments  $\sum_{i=1}^n \log P(C_i = c_i | \mathbf{X}_i = \mathbf{x}_i, \mathbf{w}, b)$  for the training set

The LR solution can be found by minimizing the logistic loss

# Support Vector Machines

As we have mentioned, it is often useful to add a regularization term of the form  $\frac{\lambda}{2} \|\mathbf{w}\|^2$ :

$$\mathbf{w}^*, b = \arg \min_{\mathbf{w}, b} \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \log \left( 1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right)$$

where  $z_i$  is the class label for sample  $\mathbf{x}_i$ , encoded as

$$z_i = \begin{cases} +1 & \text{if } c_i = \mathcal{H}_T \\ -1 & \text{if } c_i = \mathcal{H}_F \end{cases}$$

We can alternatively minimize a scaled version of the objective:

$$\mathbf{w}^*, b = \arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \log \left( 1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)} \right)$$

where  $C = \frac{1}{n\lambda}$

# Support Vector Machines

We now consider a different classifier that allows us to give a geometrical interpretation of the regularization term: the Support Vector Machine

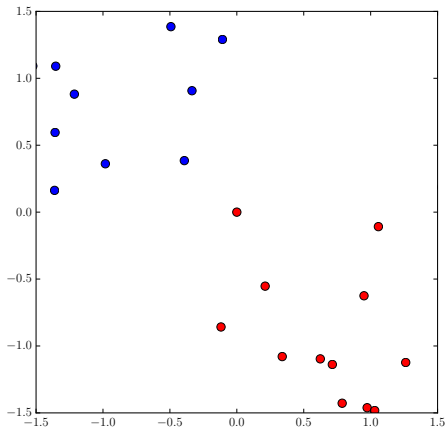
We will show that SVM can be cast as a generalized risk minimization problem

Furthermore, SVMs provide a natural way to achieve non-linear separation **without the need for an explicit expansion** of our features

In contrast with LR, however, the output of SVMs cannot be directly interpreted in terms of class posterior probabilities

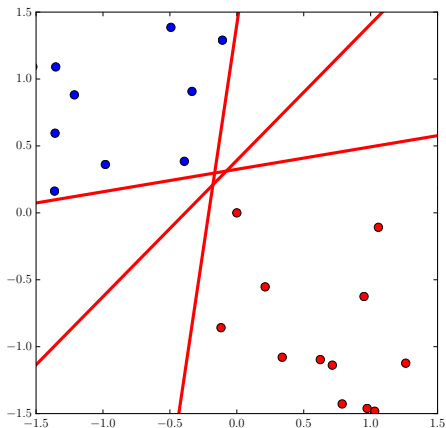
# Support Vector Machines

Assume that we have two classes that are linearly separable (i.e., we can find a linear separation hyperplane that correctly classifies all points of our training set)



# Support Vector Machines

Problem: there is an infinite number of separating hyperplanes, which one should we select?



# Support Vector Machines

We already encountered this problem with logistic regression

Logistic regression looks for the hyperplane that maximizes class probabilities

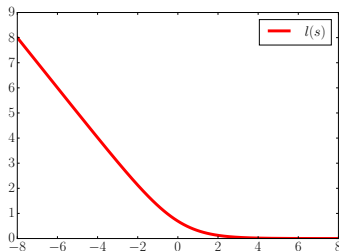
Given an hyperplane  $w$  that separates the classes, we can reduce the loss by considering an hyperplane with the same orientation, but larger norm

The model posterior probabilities will grow arbitrarily close to 1 and 0 (depending on the class) as the norm of  $w$  increases

# Support Vector Machines

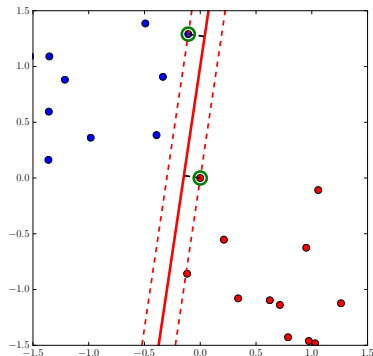
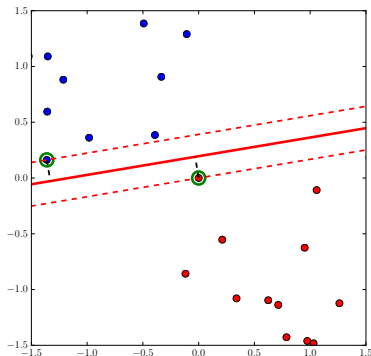
An optimal LR solution is not defined, since the objective function has an infimum for  $\|\mathbf{w}\|^2 \rightarrow +\infty$ , but not a minimum

LR optimizes the Logistic Loss. As we move further along the  $x$  axis the loss tends to 0



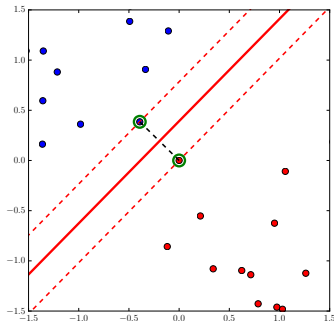
# Support Vector Machines

Intuitively, we can select the hyperplane that separates the classes with the largest **margin**



# Support Vector Machines

Intuitively, we can select the hyperplane that separates the classes with the largest **margin**



The margin is defined as the distance of the closest point w.r.t. the separation hyperplane<sup>1</sup>

---

<sup>1</sup>Some authors separately introduce the margin for each class. This distinction has no practical effects on the derivation of the SVM objective.

# Support Vector Machines

Let  $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$  be the function representing the separation surface

The distance of  $\mathbf{x}_i$  from the hyperplane is

$$d(\mathbf{x}_i) = \frac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|}$$

Let  $z_i$  denote the class for  $\mathbf{x}_i$ , with the usual encoding

$$z_i = \begin{cases} +1 & \text{if } c_i = \mathcal{H}_T \\ -1 & \text{if } c_i = \mathcal{H}_F \end{cases}$$

Since classes are separable, we consider solutions which correctly classify all points:

$$\begin{aligned} f(\mathbf{x}_i) &> 0 \text{ if } c_i = \mathcal{H}_T \\ f(\mathbf{x}_i) &< 0 \text{ if } c_i = \mathcal{H}_F \end{aligned}$$

# Support Vector Machines

The distance of  $\mathbf{x}_i$  from the hyperplane can be rewritten as

$$d(\mathbf{x}_i) = \frac{|f(\mathbf{x}_i)|}{\|\mathbf{w}\|} = \frac{|z_i(\mathbf{w}^T \mathbf{x}_i + b)|}{\|\mathbf{w}\|}$$

The maximum margin hyperplane is the hyperplane which maximizes the minimum distance of all points from the hyperplane:

$$\mathbf{w}^*, b^* = \arg \max_{\mathbf{w}, b} \min_{i \in \{1 \dots n\}} d(\mathbf{x}_i) = \arg \max_{\mathbf{w}, b} \min_{i \in \{1 \dots n\}} \frac{|z_i(\mathbf{w}^T \mathbf{x}_i + b)|}{\|\mathbf{w}\|}$$

subject to  $z_i(\mathbf{w}^T \mathbf{x}_i + b) > 0, \forall i$ .

# Support Vector Machines

For values  $\mathbf{w}, b$  which can correctly separate the classes we have  $z_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$  for all samples, and  $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$ .

We can drop the constraint by considering an **equivalent** problem

$$\begin{aligned}\mathbf{w}^*, b^* &= \arg \max_{\mathbf{w}, b} \min_i \frac{z_i(\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \\ &= \arg \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|} \min_i [z_i(\mathbf{w}^T \mathbf{x}_i + b)]\end{aligned}\tag{1}$$

We can show the two problems are equivalent:

- Hyperplanes that do not satisfy this constraint will have  $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) < 0$ , and thus cannot be an optimal solution of the second formulation (classes are separable, so an optimal solution exists with  $z_i(\mathbf{w}^T \mathbf{x}_i + b) > 0, \forall i = 1 \dots n$ )
- For hyperplanes that correctly classify all patterns, the objective functions are the same

# Support Vector Machines

Direct optimization of (1) is non trivial, thus we further transform the problem as to arrive to an easier-to-solve, **equivalent**, problem.

We can observe that the objective function in (1) is invariant under re-scaling of the parameters:

$$\frac{1}{\|\mathbf{w}\|} \min_i [z_i(\mathbf{w}^T \mathbf{x}_i + b)] = \frac{1}{\|\alpha \mathbf{w}\|} \min_i [z_i(\alpha \mathbf{w}^T \mathbf{x}_i + \alpha b)]$$

for  $\alpha > 0$  Thus, if  $(\mathbf{w}^*, b^*)$  is an optimal solution, then also  $(\mathbf{w}', b') = (\alpha \mathbf{w}^*, \alpha b^*)$  is optimal, and viceversa.

# Support Vector Machines

The collections of values  $\{(\alpha \mathbf{w}, \alpha b)\}_{\alpha \in \mathbb{R}_+}$  forms an equivalence class of equivalent solutions

For each of these equivalence classes, we are free to select any one of the equivalent solutions. In particular, we restrict our problem to solutions for which

$$\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$$

For all training points we will thus have

$$z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1 \dots n$$

# Support Vector Machines

The problem in (1) then becomes equivalent to optimizing

$$\begin{aligned} & \arg \max_{\mathbf{w}, b} \frac{1}{\|\mathbf{w}\|} \\ \text{s.t. } & \begin{cases} z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, & i = 1 \dots n \\ \min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 \end{cases} \end{aligned}$$

or, equivalently, minimizing the squared norm of  $\mathbf{w}$ :

$$\begin{aligned} & \arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t. } & \begin{cases} z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, & i = 1 \dots n \\ \min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 \end{cases} \end{aligned}$$

Finally, we observe that we can drop the last constraint, and solve the equivalent problem

$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1 \dots n \end{aligned} \quad (2)$$

Indeed, an optimal solution of (2) will automatically satisfy  $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$

In fact, if  $\mathbf{w}^*, b^*$  was a solution for which  $\min_i z_i(\mathbf{w}^T \mathbf{x}_i + b) = \psi > 1$ , then we could build the solution  $(\mathbf{w}', b') = \left(\frac{\mathbf{w}}{\psi}, \frac{b}{\psi}\right)$  which would still satisfy all constraints, and have a lower norm, thus  $\mathbf{w}^*, b^*$  would not be optimal

# Support Vector Machines

The SVM objective (primal formulation) thus consists in finding the minimizer of

$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1 \dots n \end{aligned}$$

This is a **convex quadratic programming** problem

- The objective function is convex
- The constraints form a convex set

# Support Vector Machines

To solve the SVM problem we consider a Lagrangian formulation of the problem

For each constraint we introduce the Lagrange multiplier  $\alpha_i \geq 0$ :

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [z_i (\mathbf{w}^T \mathbf{x}_i + b) - 1]$$

The optimal solution is obtained by minimizing  $L$  w.r.t.  $\mathbf{w}, b$  while requiring that either the derivatives w.r.t.  $\alpha_i$  vanish, subject to the constraints that  $\alpha_i \geq 0$ , or the corresponding  $\alpha_i$  is 0

Since the problem is convex, we can equivalently maximize  $L$  w.r.t.  $\alpha_i$  while requiring that the derivatives w.r.t.  $\mathbf{w}$  and  $b$  vanish, subject to  $\alpha_i \geq 0$

Setting the derivative of  $L$  w.r.t.  $\mathbf{w}$  and  $b$  equal to zero gives

$$\mathbf{w} = \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i$$
$$0 = \sum_{i=1}^n \alpha_i z_i$$

Replacing these constraints in  $L$  gives the *dual* SVM problem

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j z_i z_j \mathbf{x}_i^T \mathbf{x}_j$$

s.t.

$$\alpha_i \geq 0, \quad i = 1, \dots, n$$

$$\sum_{i=1}^n \alpha_i z_i = 0$$

# Support Vector Machines

The dual objective function can be expressed in matrix form as:

$$L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j z_i z_j \mathbf{x}_i^T \mathbf{x}_j = \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{H} \alpha$$

where  $\mathbf{H}$  is the matrix

$$\mathbf{H}_{ij} = z_i z_j \mathbf{x}_i^T \mathbf{x}_j$$

Matrix  $\mathbf{H}$  depends on the data only through dot-products  $\mathbf{x}_i^T \mathbf{x}_j$

As we will see shortly, this allows to extend SVMs to non-linear classification

# Support Vector Machines

For any feasible solution of the primal problem  $\mathbf{w}, b$  and any feasible solution of the dual problem  $\alpha$  we have that

$$L_D(\alpha) \leq L_P(\mathbf{w}, b)$$

where

$$L_P = \frac{1}{2} \|\mathbf{w}\|^2$$

is the primal objective

For a pair of optimal primal-dual solutions, the two values become equal (strong duality):

$$L_D(\alpha^*) = L_P(\mathbf{w}^*, b^*)$$

The difference

$$L_P(\mathbf{w}, b) - L_D(\alpha) \geq 0$$

is called **duality gap**, and is 0 for optimal solutions

# Support Vector Machines

A solution  $\mathbf{w}, b, \alpha$  will be optimal if and only if it satisfies the **Karush–Kuhn–Tucker (KKT) conditions**<sup>2</sup>

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \alpha) = \mathbf{w} - \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i = \mathbf{0}$$

$$\frac{\partial L(\mathbf{w}, b, \alpha)}{\partial b} = - \sum_{i=1}^n \alpha_i z_i = 0$$

$$z_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0 \quad \forall i$$

$$\alpha_i \geq 0 \quad \forall i$$

$$\alpha_i [z_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0 \quad \forall i$$

---

<sup>2</sup>These are both necessary and sufficient conditions for optimality of the SVM solution

# Support Vector Machines

The first and second equations encode that, at the optimal **primal** solution  $(\mathbf{w}, b)$ , the gradient of the Lagrangian with respect to  $\mathbf{w}$  and  $b$  becomes zero.

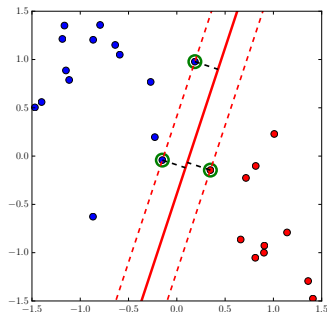
The next two equations encode that both the **primal** solution  $(\mathbf{w}, b)$  and the corresponding **dual** solution  $\alpha$  are feasible.

The last equation encodes that the optimal **dual** solution maximizes the Lagrangian. This requires that either the maximum is on the constraint, i.e.  $\alpha_i = 0$ , or the derivative of the Lagrangian is  $\frac{\partial L}{\partial \alpha_i} = 0$ . In compact form,  $\alpha_i \frac{\partial L}{\partial \alpha_i} = 0$

# Support Vector Machines

The optimal solution satisfies the Karush–Kuhn–Tucker (KKT) conditions

- For all points that do not lie on the margin (i.e.  $z_i (\mathbf{w}^T \mathbf{x}_i + b) > 1$ ) the corresponding Lagrange multiplier is  $\alpha_i = 0$
- $\alpha_i \neq 0$  implies that the corresponding point is on the margin, i.e., it is a **Support Vector**
- After we have obtained  $\alpha$ , the KKT conditions allow us to estimate  $b$



# Support Vector Machines

Since  $\mathbf{w} = \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i$ , the score for a test point  $\mathbf{x}_t$  can be computed as

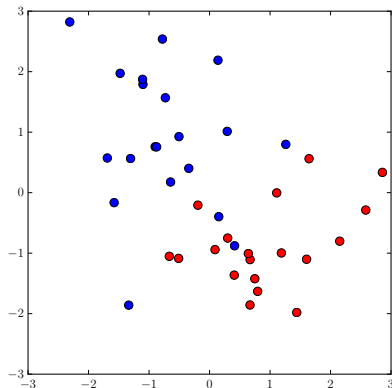
$$s(\mathbf{x}_t) = \mathbf{w}^T \mathbf{x}_t + b = \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i^T \mathbf{x}_t + b$$

Notice that, again, this depends only on dot products  $\mathbf{x}_i^T \mathbf{x}_t$

The training points that are not Support Vectors do not affect the separation surface

# Support Vector Machines

Let's consider a problem where classes are not linearly separable



# Support Vector Machines

No matter the value of  $\mathbf{w}$ , some points will violate the constraint

$$z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

We can try to minimize the number of points that violate the constraint

To do this, we introduce the **slack variables**  $\xi_i \geq 0$ , which represent how much a point is violating the constraint

We can replace the constraints  $z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$  with

$$z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

i.e. we allow training points to be inside the margin by a factor  $\xi_i$

# Support Vector Machines

We want to minimize the number of data points that lie inside the margin, i.e. the number of points for which  $\xi_i > 0$

We can consider the functional

$$\Phi(\xi) = \sum_{i=1}^n \xi_i^\sigma$$

with  $\sigma > 0$

For sufficiently small values of  $\sigma$ ,  $\Phi(\xi)$  represents the number of points inside the margin

If we remove these data points the classes become linearly separable and we can thus train a maximum margin hyperplane over the remaining points

Formally, this corresponds to the minimization of the functional

$$\frac{1}{2} \|\mathbf{w}\|^2 + CF \left( \sum_{i=1}^n \xi_i^\sigma \right)$$

s.t.

$$z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i$$

$$\xi_i \geq 0 \quad \forall i$$

where  $F$  is a monotone convex function and  $C$  is a constant

For sufficiently large  $C$  and small  $\sigma$  the optimal solution minimizes the number of points that violate the margin constraints and maximizes the margin between the remaining points

# Support Vector Machines

Unfortunately, for small values of  $\sigma$  the problem is difficult

We simplify the problem by considering  $\sigma = 1$ , which guarantees a unique solution for the separation surface, and<sup>3</sup>  $F(u) = u$

The objective function becomes

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & z_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i & \forall i = 1 \dots n \\ & \xi_i \geq 0 & \forall i = 1 \dots n \end{aligned}$$

Note that this is again a convex quadratic programming problem

---

<sup>3</sup>SVMs with  $\sigma = 2$  are sometimes used, and are usually referred to as L2-SVMs

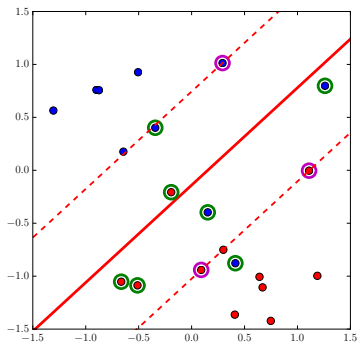
# Support Vector Machines

The terms  $\xi_i$  do not describe the number of errors, but act as a penalty

- Points inside the margin have  $\xi_i > 0$ , and miss-classified points have  $\xi_i > 1$
- The further the point is from the hyperplane, the larger the value of  $\xi_i$
- $\sum_{i=1}^n \xi_i$  is an upper bound on the number of miss-classified points (errors)
- $C$  allows selecting a trade-off between margin and errors on the training set
- The solution is often called a *soft margin* hyperplane

# Support Vector Machines

Soft margin classification:



# Support Vector Machines

As we did for the hard-margin SVM, we can introduce the Lagrangian problem

$$\begin{aligned} L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \\ = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [z_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] - \sum_{i=1}^n \mu_i \xi_i \end{aligned}$$

where  $\alpha_i \geq 0$ , and  $\mu_i \geq 0$  are an additional set of Lagrange multipliers relative to the constraints  $\xi_i \geq 0$

# Support Vector Machines

The necessary and sufficient KKT conditions are

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \mathbf{w} - \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i = \mathbf{0}$$

$$\frac{\partial L(\mathbf{w}, b, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial b} = - \sum_{i=1}^n \alpha_i z_i = 0$$

$$\frac{\partial L(\mathbf{w}, b, \boldsymbol{\alpha}, \boldsymbol{\mu})}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \quad \forall i$$

$$z_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \geq 0 \quad \forall i$$

$$\xi_i \geq 0 \quad \forall i$$

$$\alpha_i \geq 0 \quad \forall i$$

$$\mu_i \geq 0 \quad \forall i$$

$$\alpha_i [z_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] = 0 \quad \forall i$$

$$\mu_i \xi_i = 0 \quad \forall i$$

# Support Vector Machines

From the KKT conditions (1)–(3)

$$\mathbf{w} = \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i, \quad \sum_{i=1}^n \alpha_i z_i = 0, \quad \alpha_i = C - \mu_i$$

Note that  $\mu_i \geq 0$  implies that  $\alpha_i \leq C$

We can then optimize the Lagrangian w.r.t.  $\mathbf{w}$ ,  $b$  and  $\xi$  to obtain the dual problem

$$\max_{\alpha} L_D(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j z_i z_j \mathbf{x}_i^T \mathbf{x}_j$$

s.t.

$$0 \leq \alpha_i \leq C, \quad i = 1, \dots, n$$

$$\sum_{i=1}^n \alpha_i z_i = 0$$

# Support Vector Machines

Note that this problem is very similar to the hard-margin SVM

The constraint

$$\alpha_i \geq 0$$

is replaced by

$$0 \leq \alpha_i \leq C$$

Predictions are again computed as in the hard-margin case

$$s(\mathbf{x}_t) = \mathbf{w}^T \mathbf{x}_t + b = \sum_{i=1}^n \alpha_i z_i \mathbf{x}_i^T \mathbf{x}_t + b$$

Again, this depends only on **dot products**  $\mathbf{x}_i^T \mathbf{x}_t$

# Support Vector Machines

Several methods to solve the *dual* problem

Packages are available for several solvers

If we are interested in linear separation we can directly solve the *primal* SVM problem

Let's recall the primal formulation

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

s.t.

$$\begin{aligned} z_i(\mathbf{w}^T \mathbf{x}_i + b) &\geq 1 - \xi_i & \forall i \\ \xi_i &\geq 0 & \forall i \end{aligned}$$

# Support Vector Machines

For data points that are on the correct side of the margin

$$\xi_i = 0$$

while for the others we have

$$\xi_i = 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)$$

Thus the SVM primal problem can then be rewritten as

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \max [0, 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]$$

where the constraints have been absorbed by the *loss* terms

$$\max [0, 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]$$

# Support Vector Machines

Function

$$f(s) = \max(0, 1 - s)$$

is known as *hinge loss*, and is sometimes denoted as

$$f(s) = [1 - s]_+$$

We can rewrite the objective as

$$\min_{\mathbf{w}, b} \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_{i=1}^n \max [0, 1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]$$

Compare with Logistic Regression:

$$\min_{\mathbf{w}, b} \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \log [1 + e^{-z_i(\mathbf{w}^T \mathbf{x}_i + b)}]$$

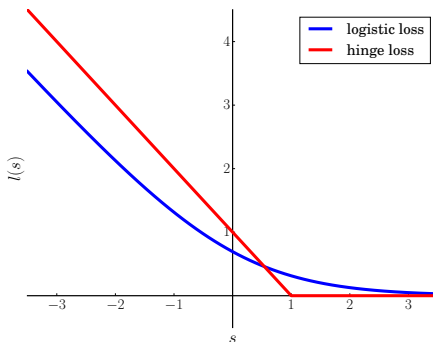
# Support Vector Machines

LR: Logistic Loss

$$l(s) = \log(1 + e^{-s})$$

SVM: Hinge Loss

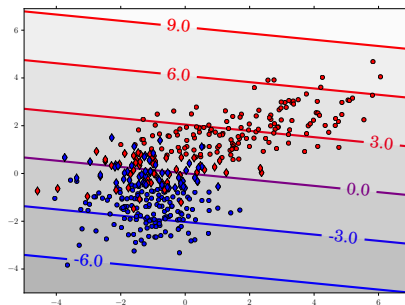
$$l(s) = \max(0, 1 - s)$$



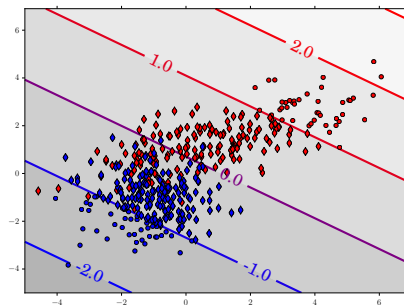
# Support Vector Machines

Binary classification:

$C = 1.0$  ( $\lambda = 0.0025$ )



$C = 0.001$  ( $\lambda = 2.5$ )



# Support Vector Machines

We have seen that we can obtain the separation hyperplane by solving either the primal or the dual problem

Primal

Minimize w.r.t.  $\mathbf{w}$  and  $b$

$$\begin{aligned} L_P(\mathbf{w}, b) &= \\ &= \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n [1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]_+ \end{aligned}$$

Dual

Maximize w.r.t.  $\alpha$

$$L_D(\alpha) = \alpha^T \mathbf{1} - \frac{1}{2} \alpha^T \mathbf{H} \alpha$$

subject to

$$0 \leq \alpha_i \leq C, \quad i = 1, \dots, n$$

$$\sum_{i=1}^n \alpha_i z_i = 0$$

For a pair of feasible solutions  $L_P(\mathbf{w}, b) - L_D(\alpha)$  is again the duality gap  
At the optimum, the gap becomes 0, i.e.  $L_P(\mathbf{w}^*, b^*) = L_D(\alpha^*)$

# Support Vector Machines

## Primal

Parameters  $\mathbf{w} \in \mathbb{R}^D$ ,  $b \in \mathbb{R}$

Scoring complexity is  $O(D)$ :

$$s(\mathbf{x}_t) = \mathbf{w}^T \mathbf{x}_t + b$$

Embedding a non-linear transformation requires explicitly defining the mapping from  $\mathbb{R}^D$  to the Hilbert space  $\mathcal{H}$

$$\Phi : \mathbb{R}^D \longrightarrow \mathcal{H}$$

## Dual

Parameters  $\alpha \in \mathbb{R}^N$

Scoring complexity is  $O(SV)$ :

$$\begin{aligned} s(\mathbf{x}_t) &= \sum_{i=1}^N \alpha_i z_i \mathbf{x}_i^T \mathbf{x}_t + b \\ &= \sum_{i|\alpha_i > 0} \alpha_i z_i \mathbf{x}_i^T \mathbf{x}_t + b \end{aligned}$$

Embedding a non-linear transformation only requires **dot-products in the expanded space**

$$\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

# Support Vector Machines

If we have a function that efficiently computes the dot-products in the expanded space

$$k(\mathbf{x}_1, \mathbf{x}_2) = \Phi(\mathbf{x}_1)^T \Phi(\mathbf{x}_2)$$

then both training and scoring can be performed by using only  $k$

Remember that, for mapping  $\Phi$ , matrix  $\mathbf{H}$  is given

$$\mathbf{H}_{ij} = z_i z_j \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

From the definition of  $k$ :

$$\mathbf{H}_{ij} = z_i z_j k(\mathbf{x}_i, \mathbf{x}_j)$$

Scoring becomes

$$s(\mathbf{x}_t) = \sum_{i=1|\alpha_i>0} \alpha_i z_i \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_t) + b = \sum_{i=1|\alpha_i>0} \alpha_i z_i k(\mathbf{x}_i, \mathbf{x}_t) + b$$

# Support Vector Machines

Function  $k$  is called **kernel function**

A kernel function allows training a SVM in a large (even infinite) dimensional Hilbert space  $\mathcal{H}$ , without requiring to explicitly compute the mapping

Even if the expansion was finite, the complexity of the primal problem may be too large

On the other hand, the complexity of the dual problem only depends on the number of training points.

In practice, we are computing a linear separation surface in the expanded space, which corresponds to a non-linear separation surface in the original feature space

# Support Vector Machines

We have already seen an example of feature expansion that provides quadratic separation surfaces, given by the mapping

$$\Phi(\mathbf{x}) = \begin{bmatrix} \text{vec}(\mathbf{x}\mathbf{x}^T) \\ \sqrt{2}\mathbf{x} \\ 1 \end{bmatrix}$$

We can compute

$$\Phi(\mathbf{x}_1)^T \Phi(\mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2)^2 + 2\mathbf{x}_1^T \mathbf{x}_2 + 1$$

It is easy verifying that the corresponding kernel is

$$k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^2$$

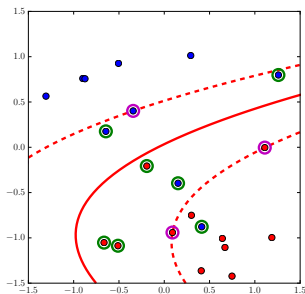
# Support Vector Machines

In general, we can define the polynomial kernels of degree  $d$  as

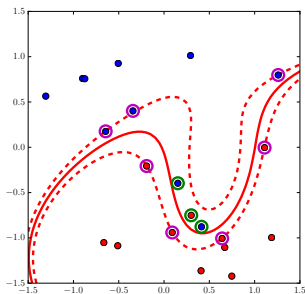
$$k(\mathbf{x}_1, \mathbf{x}_2) = (\mathbf{x}_1^T \mathbf{x}_2 + 1)^d$$

Note that the expanded space, although finite, grows very quickly

Poly —  $d = 2$ ,  $C = 1.0$



Poly —  $d = 4$ ,  $C = 10.0$



# Support Vector Machines

If we know  $\Phi$ , we can trivially define a corresponding kernel function  $k(\mathbf{x}_1, \mathbf{x}_2) = \Phi(\mathbf{x}_1)^T \Phi(\mathbf{x}_2)$

However, we typically want to avoid defining explicit mappings

We would rather like to know which functions  $k(\mathbf{x}_1, \mathbf{x}_2)$  correspond to a feature mapping  $\Phi$  in a suitable space  $\mathcal{H}$

Mercer's condition provides a sufficient condition for  $k$  to define a dot-product in an expanded space

# Support Vector Machines

Let  $k(\mathbf{u}, \mathbf{v})$  be a symmetrical function in  $L_2$

If, for all functions  $g(\mathbf{u})$  such that

$$\int g(\mathbf{u})^2 d\mathbf{u} < \infty$$

we have that

$$\int k(\mathbf{u}, \mathbf{v}) g(\mathbf{u}) g(\mathbf{v}) d\mathbf{u} d\mathbf{v} > 0$$

then  $k$  defines a dot-product in some expanded space

This condition does not tell us how to construct suitable kernels

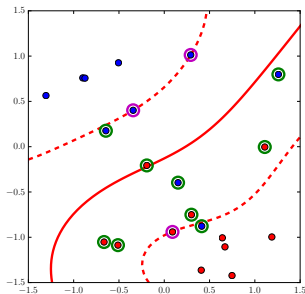
In general, we can make use of well known kernels, or use rules to combine kernel functions that guarantee that the combination is still a kernel (e.g., summing kernel functions)

# Support Vector Machines

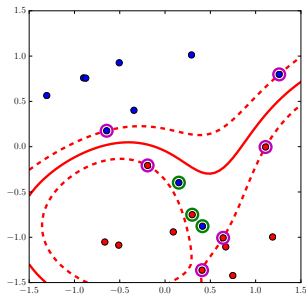
Another example of kernel function associated to an infinite-dimensional mapping (Gaussian Radial Basis Function kernel):

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$$

RBF —  $\gamma = 0.5, C = 1.0$



RBF —  $\gamma = 0.5, C = 100.0$

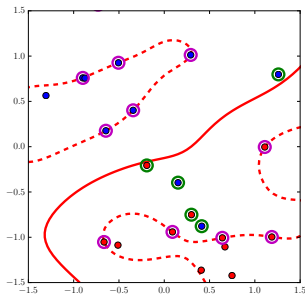


# Support Vector Machines

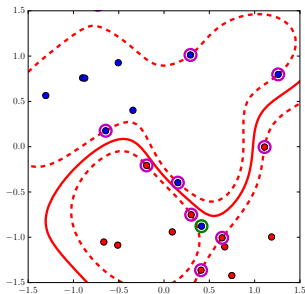
Another example of kernel function associated to an infinite-dimensional mapping (Gaussian Radial Basis Function kernel):

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$$

RBF —  $\gamma = 2.0, C = 1.0$



RBF —  $\gamma = 2.0, C = 100.0$



# Support Vector Machines

Another example of kernel function associated to an infinite-dimensional mapping (Gaussian Radial Basis Function kernel):

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$$

The kernel depends on the distance of the points

“Similar” (i.e. closer) points have higher kernel value

# Support Vector Machines

Another example of kernel function associated to an infinite-dimensional mapping (Gaussian Radial Basis Function kernel):

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-\gamma \|\mathbf{x}_1 - \mathbf{x}_2\|^2}$$

$\gamma$  defines the width of the kernel

- Small  $\gamma$ : the kernel is wide, a S.V. influences most other points
- Large  $\gamma$ : the kernel is narrow, a S.V. has very small influence on points that are not close. Assuming  $b = 0$ , we would obtain the same classification rule of 1-NN<sup>4</sup>

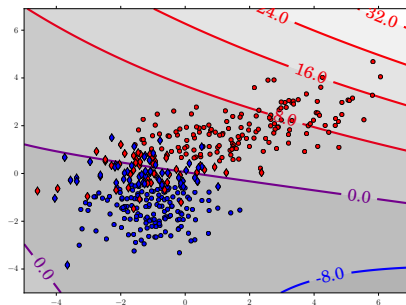
---

<sup>4</sup>In practice we should not use SVMs as replacement for K-NN, since we will incur into numerical problems due to test points all having scores  $s \approx 0$

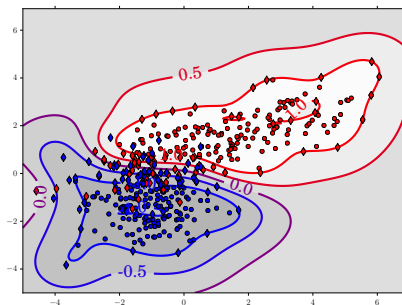
# Support Vector Machines

Binary classification:

Poly —  $d = 2$ ,  $C = 1.0$



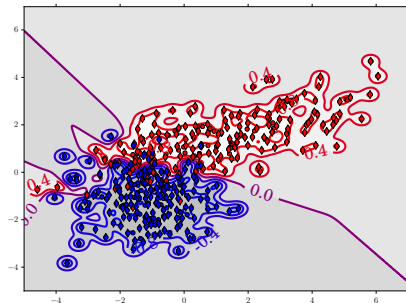
RBF —  $\gamma = 0.5$ ,  $C = 1.0$



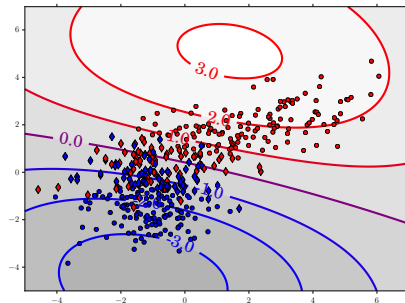
# Support Vector Machines

Binary classification:

RBF (no bias) —  $\gamma = 10$ ,  $C = 1.0$



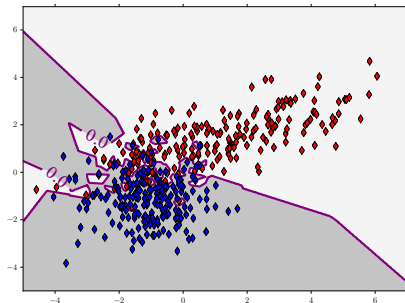
RBF (no bias) —  $\gamma = 0.02$ ,  $C = 1.0$



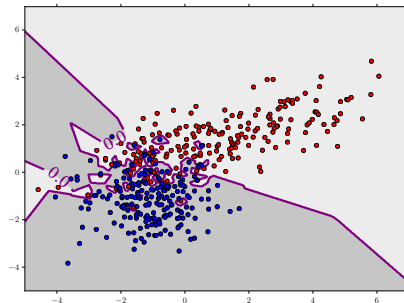
# Support Vector Machines

Binary classification:

RBF (no bias) —  $\gamma = 200$ ,  $C = 1.0$



1-NN



## Practical considerations:

- We need to take care in selecting the appropriate kernel
- Kernels often include hyper-parameters (e.g.  $\gamma$  for RBF)
  - Cross-validation can help selection of good values
- We need to select good values of the coefficient  $C$ 
  - Again, cross-validation can help

## Practical considerations:

- In some cases linear classifiers are sufficient
  - Our feature set is of sufficient high dimension to make our classes almost linearly separable

Fast algorithms for training the primal problem are available

Furthermore, linear SVM scoring can be implemented as a simple dot-product

- SVM training is not invariant under affine transformations
  - Feature pre-processing can be relevant
  - Often it is useful to center and whiten data (see the Logistic Regression slides)

## Practical considerations:

- SVM scores have no probabilistic interpretation
  - Score post-processing can be applied to estimate class posterior probabilities (e.g. we can train a generative or discriminative probabilistic model over SVM scores using an held-out dataset)
- Finally, we have to take care when training with highly unbalanced datasets (or, more precisely, when the empirical training set prior is significantly different from the target application prior). In this case, weighting the costs of different errors may be beneficial (but we still don't have a probabilistic interpretation for the score)

# Support Vector Machines

To re-balance the classes we can use a different value of  $C$  for the different classes (or even for different samples):

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n C_i [1 - z_i(\mathbf{w}^T \mathbf{x}_i + b)]_+$$

The corresponding dual problem is<sup>5</sup>

$$\begin{aligned} & \arg \max_{\alpha} \boldsymbol{\alpha}^T \mathbf{1} - \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{H} \boldsymbol{\alpha} \\ \text{s.t. } & \sum_{i=1}^n \alpha_i z_i = 0, \quad 0 \leq \alpha_i \leq C_i, \quad i = 1, \dots, n \end{aligned}$$

---

<sup>5</sup>In the laboratory we will omit the constraint related to the bias, since we will not explicitly model the bias term

# Support Vector Machines

For class balancing, we can set  $C_i = C_T$  for samples of class  $\mathcal{H}_T$  and  $C_i = C_F$  for samples of class  $\mathcal{H}_F$

We can select  $C_T = C \frac{\pi_T}{\pi_T^{emp}}$  and  $C_F = C \frac{\pi_F}{\pi_F^{emp}}$

$\pi_T^{emp}$  and  $\pi_F^{emp}$  are the empirical priors (i.e. sample proportions) for the two classes computed over the training set

In practice, we are simulating the application class proportions at training time

# Support Vector Machines

MNIST — Comparison of average pairwise EER for different kernels (raw features)

| Kernel                 | $C = 10$ | $C = 1$ | $C = 0.1$ | LogReg ( $\lambda = 1e^{-3}$ ) |
|------------------------|----------|---------|-----------|--------------------------------|
| Linear                 | 1.6%     | 1.3%    | 1.1%      | 1.2%                           |
| Poly ( $d = 2$ )       | 0.3%     | 0.3%    | 0.3%      | 0.9% <sup>6</sup>              |
| Poly ( $d = 3$ )       | 0.4%     | 0.4%    | 0.4%      | —                              |
| RBF ( $\gamma = 0.2$ ) | 0.7%     | 0.7%    | 0.9%      | —                              |
| RBF ( $\gamma = 1.0$ ) | 0.5%     | 0.5%    | 0.6%      | —                              |
| RBF ( $\gamma = 2.0$ ) | 0.1%     | 0.1%    | 0.1%      | —                              |

<sup>6</sup>Quadratic expansion of PCA-reduced features

# Support Vector Machines

SVMs provide good performance for binary classification problems

Difficult to extend to multiclass problems

Some possibilities:

- One-versus-all training
- Train over all possible pairs

In both cases we need some voting scheme to decide the actual class

# Support Vector Machines

Some possibilities (cont.):

- Introduce an objective function that maximizes the margin between each class and all the others. For example, (we ignore the bias terms)

$$\min_{\mathbf{W}} \frac{1}{2} \|\mathbf{W}\|_2^2 + C \sum_{i=1}^n \max_{y'} [(\mathbf{w}_{y'} - \mathbf{w}_{z_i})^T \mathbf{x}_i + \delta_{y', z_i}]$$

Compare with multiclass logistic regression

$$\min_{\mathbf{W}} \frac{\lambda}{2} \|\mathbf{W}\|_2^2 + \frac{1}{n} \sum_i \log \left( \sum_{y'} e^{(\mathbf{w}_{y'} - \mathbf{w}_{z_i})^T \mathbf{x}_i} \right)$$

Kernel methods are not restricted to SVMs

- Logistic Regression can be extended to incorporate kernels
- In general, we can apply the kernel tricks to problems which depend only on dot-products
  - Kernel PCA
  - Gaussian Processes
  - ...